Low Voltage Individual Technical Mission - Answers

Youssef Sameh - ASU Racing Team, Low Voltage sub-team

---

1. Competition Rules

Brake System Plausibility Device (BSPD) - Rule T11.6 This is a circuit that checks for something that should never happen at the same time: hard braking and the motors still getting a lot of power. It reads the brake pressure sensor, and if it sees hard braking (around 30 bar, calibrated so the wheels don't lock) while more than 5kW is still going to the motors, it opens the shutdown circuit and cuts the power. It's built from normal analog components, not code, because its whole job is to catch a failure in the software or the throttle sensor - so it can't depend on that same software to work. It needs to see the fault for more than 500ms before it trips, so a short glitch doesn't shut the car down for no reason.

Insulation Monitoring Device (IMD) - Rule EV6.3 The high voltage system in the car isn't connected to the car's ground, on purpose, so one single fault to ground on its own isn't dangerous. The IMD keeps checking the resistance between the HV wiring and the chassis. If that resistance drops too low, it means there's a real chance of someone getting shocked, so it opens the shutdown circuit right away, without any microcontroller involved. It's basically the main thing protecting people from a hidden insulation fault they can't see.

Accumulator Isolation Relay (AIR) - Rule EV5 These are the actual relays inside the battery box that physically disconnect the + and - wires coming out of the accumulator. Everything else in this list works by opening the shutdown circuit, but it's the AIRs that actually do the disconnecting once that happens. So the rest of the car is only ever live when the AIRs are closed, which is the whole point of the safety design. (Worth double checking the exact sub-clause number for this one in your own copy of the rulebook, it shifts a bit between years.)

Brake Over-Travel Switch (BOTS) - Rule T6.2 A mechanical switch behind the brake pedal. If one of the two brake circuits fails, the pedal travels further than it normally would, and that extra travel hits the switch and opens the shutdown circuit. Pressing it again and again doesn't reset it, and the driver can't reset it from the seat either - someone from the team has to check the car first before it can go again.

Inertia Switch - Rule T11.5 This trips when the car suddenly decelerates the way it would in a crash (something like 6 to 11 g). It shuts off the tractive system automatically even if nobody manages to hit a shutdown button. Like the BOTS, it latches, meaning it stays tripped until someone resets it by hand, so the car can't get re-energized right after a crash without someone deliberately doing it.

Tractive System Active Light (TSAL) - Rule EV4.10 A light on the outside of the car that shows whether the high voltage is actually on. It's built with plain hardwired electronics that check the real voltage on the HV bus itself, not just whether the system was told to turn on. Solid red means the TS is active, don't touch it. Flashing red means it's active but something isn't confirming properly. Green means it's safe.

---

2. Crosstalk in PCBs

Crosstalk is when a signal on one trace leaks onto a nearby trace that isn't even

connected to it, through the electric and magnetic fields between them. It gets worse the longer two traces run close and parallel to each other, and the faster the signal switches.

Ways to reduce it: leave more space between the traces, keep a solid ground plane right under them, put a grounded trace between two sensitive lines if they have to run close together, don't let traces run parallel for long distances, cross traces at 90 degrees instead of running alongside each other, and keep the traces close to the ground plane underneath them so the field stays tight instead of spreading sideways.

---

3. Circuit Question - Pull-down Resistor and Noise

(Still needs to actually be built and simulated in Proteus for the screenshots - this is just what the analysis says you should see.)

a) What the circuit does: It shows why a digital input needs a resistor holding it at a known level, instead of being left floating. R1 holds the node at 0V when the switch is open, and the switch pulls it up to 5V when pressed.

b) The dashed box: The 50Hz source with R3 (1G ohm) is just modeling stray noise coupling into the circuit, like mains hum picking up on a wire near AC power. The huge resistance is how the sim represents a very weak, indirect coupling path, not a real wired connection.

c) About R1: 1. If you remove R1, the node has nothing holding it to ground when the switch is open, so it becomes floating. Now the 1G-ohm noise source is basically the only thing driving it, so the node picks up the 50Hz noise instead of sitting cleanly at 0V. 2. A node like that (no resistor holding it to a level) is called a floating input. 3. In real life R1 should be small enough to override any noise or leakage current and hold a clean level, but big enough that it doesn't waste much current when the switch closes. Something like 1k to 10k ohms is typical for a 5V logic input.

d) The arrow and R2: The arrow points to the actual node that gets read, basically what a microcontroller pin would see. R2 (100M ohm) represents the input resistance of that pin - a real digital input isn't a perfect open circuit, it has some very high resistance.

e) If the active signal were 0V instead of 5V: You'd swap it to a pull-up instead of a pull-down - move the resistor to +5V and have the switch pull the node down to ground when pressed. Normally it reads high, and pressing the switch makes it read low. That's called a pull-up configuration.

f) Touching the circuit (bonus): Your body picks up the same 50/60Hz mains hum from the air around you, so bringing your hand near or touching an unshielded, high-impedance node couples that noise straight into it. It's the same reason a floating scope probe shows a noisy 50/60Hz wave the second you touch the tip.

---

4. Decoupling Capacitors vs Bulk Capacitors

Both keep a power rail stable, just at different speeds.

Decoupling (bypass) capacitors are small, usually 10nF to 100nF, and go right next to each IC's power pins. Chips pull current in very fast little bursts every clock cycle, and even a short bit of PCB trace has enough inductance that the main power supply can't react fast enough. The small cap sits right there and supplies that quick burst locally instead.

Bulk capacitors are much bigger, tens to thousands of microfarads, usually near a regulator or where power comes onto the board. They handle the slower, bigger swings

in current, like a motor starting up or a burst of digital activity, and smooth out ripple from a regulator. They're too slow (higher parasitic inductance) to react to the nanosecond-scale switching a decoupling cap handles, which is why you use both together.

---

5. Differential Pairs

A differential pair is two traces carrying the same signal as opposite voltages (one goes up while the other goes down). CAN H/L is an example. The receiver only cares about the difference between them, so any noise that lands equally on both traces gets cancelled out, and since the currents run opposite directions their magnetic fields mostly cancel too, which cuts down EMI.

For it to actually work you need to: keep the width and spacing of the two traces the same the whole way (for a consistent impedance), keep both traces the same length so there's no timing skew, keep them close together and never let them split apart or run on different layers for long, route away from other noisy signals so nothing couples unevenly onto just one of the two traces, keep a solid ground plane underneath the whole pair, and avoid sharp 90 degree bends.

---

6. Polygon Pours

A polygon pour is just a big filled area of copper on a layer, connected to one net (usually ground or a power rail) instead of leaving that space empty or routing a thin trace through it.

It's used for power and ground because a wide sheet of copper has way less resistance and inductance than a thin trace, so it carries current with less voltage drop and heating. It also gives signal traces routed above or below it a clean, solid return path, which cuts down crosstalk and EMI. It helps spread heat away from hot parts too. And practically, it saves time since the tool fills the pour automatically to any pad on that net instead of you having to route a wide trace to every single pin.

---

7. Stepping 0-12V Down to 0-5V on a 6mW Budget

The cheapest option with no active parts is just two resistors as a voltage divider.

Total resistance needed from the power limit: R_total = 12^2 / 0.006 = 24,000 ohms

Ratio needed so 12V maps to 5V: R_bottom = 24k * (5/12) = 10k ohms R_top = 24k - 10k = 14k ohms

Check: at 12V in, output = 12 * (10k/24k) = 5V, and power = 12^2/24k = 6mW. Matches exactly.

a) Why these values: The total (24k) comes straight from the power limit at the worst case, 12V. The split between the two resistors (10k and 14k) comes from needing 12V in to give exactly 5V out, so the whole sensor range fits the whole ADC range with nothing clipped or wasted. (In practice you'd round to standard resistor values, like 15k and 10k, and the numbers shift a little.)

b) With the faulty ADC pin pulling 0.5mA: The divider's output resistance (both resistors in parallel) is (14k*10k)/24k, about 5.83k ohms. That leakage current pulls an extra voltage drop of 0.5mA* 5.83k, about 2.9V, off the reading. That's more than half the full 5V range, so the ADC would read a badly wrong, low value no matter what the sensor

is actually showing. You'd simulate this by adding a 0.5mA current sink at the output node and watching the voltage drop.

c) Max acceptable leakage: If you want the error to stay under about 1% of full scale (50mV), the max leakage would be 0.05V / 5.83k, around 8.6 microamps. That's way smaller than the 0.5mA fault, which is why that fault causes such a big problem. This is really the point of the question: a resistor divider sized for a tiny power budget ends up with a high output impedance, so it's very sensitive to any leakage on the ADC pin. A real design would usually add an op-amp buffer between the divider and the ADC to fix this.

---

8. Two Interrupts at the Same Time

The processor doesn't actually run both at once, it decides which one goes first based on priority. Every interrupt source has a priority level, and whichever pending interrupt has the higher priority gets serviced first. The other one just stays pending and gets handled right after (or can even interrupt the first one mid-way if nesting is allowed and it has higher priority). If both interrupts happen to have the exact same priority, the hardware falls back on a fixed order, so there's never actually a random or unclear outcome.

Through ARM (bonus): On ARM Cortex-M chips like the STM32, this is handled by the NVIC. Each interrupt has a priority value, split into a group priority and a sub-priority depending on how priority grouping is set. If two interrupts are pending at once, the NVIC checks group priority first (lower number wins), then sub-priority if those are equal, and if everything is exactly equal it just goes by the fixed position of the interrupt in the vector table. The one that loses out doesn't get skipped, it stays pending and gets handled right after the first one finishes, with very little extra delay because of a feature called tail-chaining.

---

9. Communication Protocols in Embedded Systems

A communication protocol is just an agreed set of rules for how two devices talk to each other - things like the voltage levels used, whether there's a shared clock, how a message is framed, and how errors get caught. It's what lets a microcontroller talk to a sensor or another chip from a totally different manufacturer and still understand it.

Three examples: - UART: asynchronous, no shared clock, both sides just agree on a baud rate. Simple, good for debug output, GPS modules, basic sensors. - SPI: synchronous, full duplex, uses separate MOSI, MISO, SCK lines plus a chip-select per device. Fast and simple, good for things like SD cards, displays, or talking between two microcontrollers. - I2C: synchronous, two wires (SDA, SCL), supports multiple devices on the same bus using addresses instead of separate select lines. Slower than SPI but uses way fewer pins, good for connecting several low-speed sensors.

Worth mentioning CAN too, since it's what's actually used on the car between the ECUs, BMS, and DAQ - differential, multi-master, and built to be reliable in a noisy electrical environment.

---

10. STM32F4 vs Raspberry Pi Zero vs ATmega

STM32F4 is a 32-bit ARM Cortex-M4 microcontroller, no operating system, running up to around 168-180MHz, with a lot of built-in peripherals (multiple ADCs, timers, SPI, I2C, UART, CAN, USB). Its main strength is that everything happens in a predictable number

of clock cycles, since there’s no OS scheduler getting in the way. That’s why it’s the right choice for real-time stuff: motor control, precise timing, safety circuits, exactly the kind of work this assignment is about.

Raspberry Pi Zero is a full computer, not a microcontroller. It runs actual Linux, has way more RAM and storage, and can do networking, camera input, that kind of thing. It’s good for heavier, non-time-critical work like vision processing or logging data to a file. The downside is Linux introduces timing jitter, so it’s not something you’d trust for a hard real-time control loop or a safety interlock.

ATmega (like the classic Arduino chip) is an 8-bit microcontroller, much slower and simpler than the STM32F4, with less memory and fewer peripherals. What it’s good at is being cheap, low power, and easy to program. Fine for basic tasks like reading a sensor or blinking an LED, but not enough headroom for something like fast multi-channel ADC sampling plus CAN plus telemetry all at once.

So basically: STM32F4 for real-time control, Raspberry Pi Zero for heavier computing that doesn’t need to be real-time, and ATmega for simple, cheap, low-power tasks.

---

11. Reading Euler Angles from the BNO055

Once the BNO055 is set to a fusion mode (NDOF is the usual one, combining the accelerometer, gyroscope and magnetometer), it keeps computing Heading, Roll and Pitch on its own and puts them in registers you can read over I2C.

Steps: 1. Set the mode register (OPR_MODE, address 0x3D) to NDOF (0x0C) so it starts computing orientation. 2. Read the six Euler angle registers as three pairs: 0x1A/0x1B for Heading, 0x1C/0x1D for Roll, 0x1E/0x1F for Pitch. 3. Combine each LSB/MSB pair into a signed 16-bit number, then divide by 16 to get degrees, since the default scale is 1 degree = 16 LSB.

Documentation: Bosch BNO055 datasheet (BST-BNO055-DS000), at https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bno055-ds000.pdf - the register addresses are in section 4.3, items 4.3.27 to 4.3.32, pages 66-67.

That’s everything except the two hands-on projects (Q12 Altium PCB, Q13 STM32 SPI milestones), and Q3/Q7 still need to actually be built and simulated in Proteus for the real screenshots.